



BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

Facultad de Ciencias de la Computación

Licenciatura en Ingeniería en Ciencias de la Computación

TUTORA: DRA. ANABEL HERNÁNDEZ RAMÍREZ

NOMBRE: DAYANA PÉREZ FLORES

MATRICULA: 202141156

NRC: 50326

PROGRAMA DE PRÁCTICA PROFESIONAL: *SISTEMA DE
ALIMENTACIÓN ASISTIDA MEDIANTE BRAZO ROBÓTICO Y VISIÓN
POR COMPUTADORA*

RESPONSABLE DEL PROYECTO: DR. JESÚS RICARDO LÓPEZ
GUTIÉRREZ

FOLIO DEL PROGRAMA: 231093

DEPENDENCIA: INSTITUTO NACIONAL DE ASTROFÍSICA, ÓPTICA
Y ELECTRÓNICA (INAOE)

FECHA DE INICIO: 6 DE ENERO DE 2026

FECHA DE FINALIZACIÓN: 6 DE JULIO DE 2026

HORAS TOTALES: 520 HORAS

FECHA: 12 DE AGOSTO DE 2026

ÍNDICE

1. Introducción.....	4
2. Planteamiento del problema	4
3. Objetivos.....	5
3.1. Objetivo general	5
3.2. Objetivos específicos.....	5
4. Marco teórico.....	5
4.1. Vision por computadora.....	5
4.2. Cámaras de profundidad RGB-D	5
4.3. Segmentación de imágenes.....	6
4.4. Modelos fundacionales	6
4.4. YOLO (<i>You Only Look Once</i>)	6
4.5. SAM (<i>Segment Anything Model</i>).....	6
4.6. SAM 3 y Segmentación Semántica	7
4.7. ROS (Robot Operating System)	7
4.8. Árbol de transformaciones espacial-temporales (TF)	7
4.9. Transformaciones homogéneas	7
4.10. Cinemática de manipuladores o brazos robóticos y cadenas cinemáticas.....	7
4.11. Calibración cámara-robot (<i>Hand-Eye Calibration</i>).....	8
4.12. Marcadores ArUco.....	8
5. Metodología.....	8
6. Desarrollo de la practica profesional	9
6.1. Calibración	9
6.2. Segmentación	10
6.2.1. Segmentación híbrida mediante YOLOv8s y SAM2_b	11
6.2.2. Segmentación mediante SAM 3 (<i>Segment Anything with Concepts</i>).....	12
6.2.2.1. Arquitectura general del nodo.....	12
6.2.2.2. Filtrado geométrico y discriminación de máscaras (MaskFilter)	13
Filtros básicos de escala, proporción, compacidad y contención jerárquica	14
6.2.2.3. Preprocesamiento de imagen y normalización fotométrica.....	16
Transformación al espacio de color CIE LAB	17

Ecualización adaptativa del histograma (CLAHE)	17
Reconstrucción y almacenamiento temporal para inferencia	18
6.2.2.4. Criterio adaptativo de selección del cubo objetivo.....	18
Extracción de centroides.....	18
Cálculo del centro del grupo.....	19
6.2.2.5. Mapeo de profundidad y proyección 3D	20
Filtrado espacial de la nube de puntos.....	20
Proyección perspectiva inversa de puntos 3D	20
Mapeo y muestreo sobre la máscara binaria.....	20
Estimación de profundidad representativa y coordenadas 3D.....	21
6.2.2.6. Publicación de los resultados en tópicos de ROS.....	22
Publicación del centroide objetivo	22
Publicación de la nube de puntos segmentada.....	22
6.3. Transformación de coordenadas al espacio del robot.....	22
6.3.1. Configuración ojo en mano (<i>Hand-in-Eye</i>)	22
6.3.2. Formulación matemática y árbol de transformaciones (TF)	23
6.3.3. Implementación algorítmica en ROS	23
6.3.4. Sincronización temporal y publicación para la planeación de agarre	23
7. Resultados obtenidos	24
7.1. Desempeño del módulo de percepción y segmentación.....	24
7.2. Validación de la lógica de selección y transformación espacial	24
7.3. Integración con el sistema control y guiado	25
7.4. Oportunidades de Mejora y Trabajo Futuro	25
8. Conclusiones.....	25
Referencias	26
Anexo	27

1. Introducción

La pérdida de movilidad en las extremidades superiores genera limitaciones funcionales severas en la vida cotidiana de una persona. Esta restricción de movimiento implica una dependencia constante de ayuda para realizar actividades básicas, como alimentarse por sí mismo, lo que representa un gran desafío para quienes viven con estas discapacidades. Por ello, se han planteado múltiples investigaciones y proyectos para la asistencia a dichas tareas. En este caso, se propone automatizar la asistencia de alimentación, sin embargo, esta tarea presenta importantes desafíos computacionales y mecánicos, pues requiere reconocer e aislar objetos deformables, irregulares y altamente apilados (tales como cubitos de fruta dentro de un recipiente) bajo condiciones cambiantes de iluminación, proyectar espacialmente dichos objetivos al espacio global de trabajo e integrar una línea de trabajo de visión con planificadores cinemáticos y máquinas de estados en tiempo real.

Este documento presenta el reporte final de las prácticas profesionales realizadas en el Área de Electrónica del Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE), dentro del proyecto “*Sistema de alimentación asistida mediante brazo robótico y visión por computadora*”. El objetivo central del proyecto fue diseñar, integrar y evaluar un flujo completo de visión artificial y control cinemático que permitiera a un brazo robótico Kinova Gen2, tanto en simulación como de forma física, localizar y aislar porciones pequeñas de alimento (cubitos de fruta) dentro de un recipiente, para automatizar tareas de asistencia alimentaria a personas con movilidad reducida.

El proyecto se desarrolló a lo largo de 520 horas. Durante los primeros tres meses de la estancia se completó la configuración óptica del sistema y la calibración mano-ojo (hand-eye) entre la cámara Intel RealSense D415 y el brazo Kinova, además de un primer prototipo de detección basado en la combinación de YOLOv8 y SAM 2. El presente reporte final resume brevemente esa primera etapa y profundiza en el trabajo realizado durante los últimos tres meses, la migración a una arquitectura de segmentación más robusta basada en SAM 3, la versión definitiva del nodo de transformación de coordenadas, y la integración completa del sistema, en conjunto con el módulo de planeación de trayectorias y la máquina de estados desarrollada por otra persona, hasta lograr la ejecución exitosa de la rutina completa de alimentación asistida.

2. Planteamiento del problema

Las personas con movilidad reducida en las extremidades superiores frecuentemente dependen de un cuidador para realizar tareas cotidianas tan básicas como alimentarse. La automatización de esta tarea mediante un brazo robótico requiere resolver, de manera confiable y en tiempo real, problemas técnicos fundamentales, tales como percibir con precisión la posición de porciones de alimento pequeñas y de forma irregular dentro de un recipiente, bajo condiciones de iluminación variables; traducir esa percepción, dada en el marco de referencia de la cámara, al marco de referencia del brazo, de forma dinámica y con

la menor latencia posible; y ejecutar de forma segura la secuencia completa de aproximación, sujeción y traslado del alimento.

3. Objetivos

3.1. Objetivo general

Diseñar, implementar e integrar un sistema de visión por computadora y control cinemático que permita a un brazo robótico Kinova Gen2 localizar, segmentar y alcanzar porciones individuales de alimento dentro de un recipiente, tanto en un entorno simulado (Gazebo) como en la plataforma física, como base de un sistema de alimentación asistida.

3.2. Objetivos específicos

- Calibrar los parámetros extrínsecos entre la cámara Intel RealSense D415 y el brazo robótico Kinova Gen2 mediante un procedimiento de calibración hand-eye.
- Diseñar un pipeline de segmentación semántica capaz de aislar, a nivel de píxel, cubitos de fruta individuales bajo condiciones de iluminación variables.
- Implementar un nodo de transformación de coordenadas que traduzca, en tiempo real, los centroides detectados por la cámara al marco de referencia de la base del robot.
- Integrar el sistema de percepción con el módulo de planeación de trayectorias y la máquina de estados del brazo robótico, validando su funcionamiento en simulación y de forma física.

4. Marco teórico

A continuación, se describen los fundamentos conceptuales y tecnológicos sobre los cuales se construyó el sistema desarrollado durante la práctica profesional.

4.1. Vision por computadora

La visión por computadora es la rama de la inteligencia artificial orientada a dotar a los sistemas computacionales de la capacidad de procesar, interpretar y extraer información cuantitativa y cualitativa a partir de datos visuales, como imágenes o secuencias de video. En el ámbito de la robótica de manipulación, este componente resulta indispensable, pues proporciona al robot la percepción necesaria para determinar la posición y orientación de los objetos en su entorno en tiempo real, evitando la dependencia de coordenadas preprogramadas o entornos estrictamente estructurados.

4.2. Cámaras de profundidad RGB-D

Para implementar la percepción visual en tareas de manipulación, se emplean cámaras de profundidad RGB-D, como el sensor Intel RealSense D415. Estos dispositivos complementan la información cromática de una imagen convencional (RGB) con un mapa

de distancias por píxel (canal D). En la cámara D415, la profundidad se obtiene mediante un sistema estéreo de sensores infrarrojos asistido por un proyector de luz infrarroja que incrementa la textura de la escena para mejorar la correspondencia estéreo (Carfagni et al., 2019).

Esta información permite reconstruir, para cada píxel de interés, una coordenada tridimensional (X, Y, Z) respecto al sistema de referencia de la cámara, mediante el modelo de proyección de cámara *pinhole* y sus parámetros intrínsecos: las distancias focales (f_x, f_y) y el punto principal (c_x, c_y) . En el rango de trabajo típico de tareas de manipulación de corto alcance, este tipo de sensores ofrece una precisión metrológica en el canal de profundidad del orden del milímetro (Carfagni et al., 2019; Intel Corporation, 2023).

4.3. Segmentación de imágenes

La segmentación de imágenes consiste en clasificar cada píxel de una imagen digital asignándole una etiqueta semántica correspondiente al objeto o región a la que pertenece. A diferencia de la detección de objetos, la cual únicamente delimita las regiones mediante cuadros delimitadores (*bounding boxes*), la segmentación proporciona una delimitación precisa de la geometría y los bordes de los elementos en la escena. En particular, la segmentación semántica asocia cada píxel con una categoría o concepto visual específico (por ejemplo, identificando y diferenciando clases de objetos en el entorno).

4.4. Modelos fundacionales

Los modelos fundacionales de visión son arquitecturas de aprendizaje profundo entrenadas sobre volúmenes masivos de datos a gran escala, lo que les otorga una alta capacidad de generalización ante clases de objetos no vistas durante su entrenamiento (*zero-shot learning*). En tareas de manipulación robótica, el uso de estos modelos permite segmentar objetos de geometría irregular o deformables sin necesidad de reentrenar la red neuronal para cada tipo de elemento.

4.4. YOLO (*You Only Look Once*)

Familia de redes neuronales convolucionales diseñadas para la detección de objetos en tiempo real. Su versión YOLOv8 incorpora una cabeza de detección desacoplada y libre de anclas (*anchor-free*), lo que incrementa la precisión y velocidad respecto a versiones previas (Jocher et al., 2023).

4.5. SAM (*Segment Anything Model*)

Familia de modelos fundacionales de segmentación desarrollada por Meta AI Research. Las versiones iniciales, SAM (Kirillov et al., 2023) y SAM 2 (Ravi et al., 2024), introdujeron la segmentación interactiva (*promptable*) mediante puntos, cajas o video, pero requerían de una pista geométrica previa o un detector externo como YOLO para localizar los objetos.

4.6. SAM 3 y Segmentación Semántica

La tercera generación, SAM 3 (Meta AI, 2025), introduce la segmentación semántica basada en conceptos (*Promptable Concept Segmentation*). A través del predictor semántico de Ultralytics (SAM3SemanticPredictor), el modelo es capaz de recibir descripciones o etiquetas en texto libre de alto nivel e identificar y segmentar automáticamente todas las instancias pertenecientes a dicha categoría semántica.

4.7. ROS (Robot Operating System)

Robot Operating System (ROS) es un framework de código abierto diseñado para el desarrollo de software robótico que provee una abstracción de hardware y una capa de comunicación entre procesos distribuidos, denominados nodos. Esta comunicación se realiza mediante un esquema de publicación-suscripción a través de tópicos, llamadas a servicios y mensajes estandarizados.

4.8. Árbol de transformaciones espacial-temporales (TF)

Un componente fundamental dentro del ecosistema ROS es la librería tf2, encargada del mantenimiento del árbol de transformaciones (TF) del robot. El TF es un grafo dinámico estructurado en forma de árbol con buffer temporal, el cual rastrea y almacena continuamente las relaciones de posición y orientación (transformaciones homogéneas) entre los distintos sistemas de referencia (frames) de los eslabones (links) del brazo y los sensores del entorno (Foote, 2013).

4.9. Transformaciones homogéneas

La posición y la orientación de un cuerpo rígido en el espacio euclidiano tridimensional se pueden representar de forma compacta mediante una matriz de transformación homogénea $T \in \mathbb{SE}(3)$ de dimensión 4×4 . Esta matriz integra en una sola estructura una submatriz de rotación $R \in \mathbb{SO}(3)$ de 3×3 y un vector de traslación $t \in \mathbb{R}^3$ de 3×1 :

$$T = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

La propiedad fundamental de estas matrices radica en que permiten encadenar cambios entre distintos sistemas de referencia coordinados mediante la multiplicación matricial sucesiva. De este modo, si se conoce la transformación de un marco A respecto a un marco B (T_B^A) y la de un marco B respecto a un marco C (T_C^B), la transformación directa del marco A al marco C se obtiene como $T_C^A = T_B^A \cdot T_C^B$.

4.10. Cinemática de manipuladores o brazos robóticos y cadenas cinemáticas

La cinemática de manipuladores es la rama de la robótica que analiza la descripción geométrica del movimiento del robot con respecto a un sistema de referencia fijo, sin considerar las fuerzas o momentos que causan dicho movimiento. Un manipulador robótico

se modela como una cadena cinemática abierta compuesta por una secuencia de eslabones rígidos unidos mediante articulaciones.

4.11. Calibración cámara-robot (*Hand-Eye Calibration*)

La calibración hand-eye determina la transformación homogénea rígida y fija entre el sistema de referencia de una cámara montada sobre el brazo y el eslabón donde se soporta. Uno de los revolvedores más utilizados es el algoritmo de Daniilidis (1999), basado en cuaterniones duales, el cual calcula simultáneamente la rotación y la traslación relativas a partir de pares de poses sincronizadas del robot y de un patrón visual de marcadores ArUco, minimizando el error de consistencia geométrica.

4.12. Marcadores ArUco

Los marcadores ArUco son patrones binarios cuadrados que codifican un identificador único y permiten una detección robusta y de bajo costo computacional (Garrido-Jurado et al., 2014). A partir de las esquinas del marcador en la imagen 2D y de los parámetros intrínsecos de la cámara, se estima su pose 3D (rotación y traslación) respecto al sensor resolviendo el problema *Perspective-n-Point* (PnP) mediante la función `cv2.solvePnP` de OpenCV, la cual minimiza el error de reproyección entre el modelo tridimensional conocido y su proyección observada (Lepetit et al., 2009).

5. Metodología

El desarrollo del proyecto siguió una metodología iterativa e incremental, dividida en fases secuenciales que fueron validadas primero en simulación (entorno Gazebo, réplica virtual del brazo Kinova Gen2 y la cámara D415) y posteriormente en la plataforma física del laboratorio, minimizando así el riesgo de daños al hardware durante las pruebas iniciales de cada componente. Las herramientas principales empleadas fueron: ROS Noetic como middleware de comunicación entre nodos; Python 3 como lenguaje de desarrollo; OpenCV para el procesamiento de imágenes y la calibración; la librería Ultralytics para los modelos YOLOv8 y SAM 3; y Gazebo como entorno de simulación física.

De forma general, la metodología de trabajo se estructuró en las siguientes fases:

- Fase I — Configuración óptica y calibración: puesta a punto de la cámara RGB-D y calibración hand-eye respecto al brazo Kinova.
- Fase II — Percepción y segmentación: diseño e iteración del pipeline de detección y segmentación de objetos, culminando en la arquitectura basada en SAM 3.
- Fase III — Transformación de coordenadas: desarrollo del nodo encargado de expresar las detecciones de la cámara en el marco de referencia del robot.

- Fase IV — Integración y pruebas: acoplamiento con el módulo de planeación de trayectorias y la máquina de estados, y validación de la rutina completa en simulación y físicamente.

6. Desarrollo de la practica profesional

6.1. Calibración

Durante la primera fase del proyecto, el objetivo principal fue determinar la matriz de calibración extrínseca entre la cámara de profundidad Intel RealSense D415 y el eslabón `m1n6s300_link_5` del brazo robótico sobre el cual se encontraba montada. En los experimentos iniciales se desarrolló un script de calibración diseñado para procesar una plantilla compuesta por 12 marcadores ArUco organizados en un tablero. Sin embargo, esta primera implementación presentó deficiencias metodológicas, ya que, por un lado, el algoritmo tomaba únicamente el primer marcador detectado de forma individual para estimar la pose en lugar de considerar la geometría global del patrón; por otro lado, la disposición totalmente plana y horizontal del objetivo limitaba la variación en la orientación espacial y el rango de movimiento sobre el eje Z del sensor, generando ambigüedades en la estimación de la profundidad. Esto provocó lecturas inestables, con alta dispersión angular y fluctuaciones en la traslación que impidieron obtener una calibración convergente del sistema.

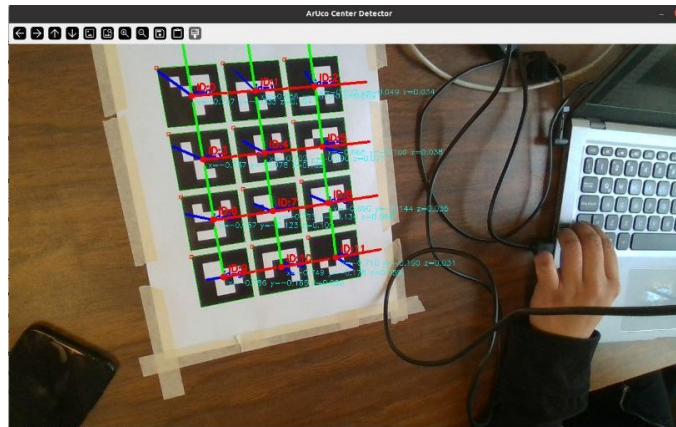


Figura 1. Detección y estimación de pose de los marcadores ChArUco durante la calibración hand-eye.

Para solucionar estas limitaciones, se reestructuró la metodología de captura e inclinación del objetivo visual. En lugar de un tablero plano extenso, se optó por emplear un único marcador ArUco de tamaño conocido (10.8 cm) con identificador exclusivo `id = 0`, fijado sobre una superficie rígida con una inclinación espacial libre y no coplanar con respecto a la base del robot (`m1n6s300_link_base`). Esta inclinación intencional rompió la degeneración geométrica en los planos paralelos y permitió orientar la cámara en un espacio de trabajo más amplio durante las tomas.

El nodo de calibración final se implementó en ROS Noetic utilizando un hilo de ejecución secundario para solicitar la captura interactiva tras posicionar manualmente el brazo robótico. En cada una de las 20 poses capturadas, el *script* filtró únicamente las detecciones válidas del marcador $id = 0$, calculando su pose 3D con `cv2.aruco.estimatePoseSingleMarkers` y extrayendo simultáneamente la transformación homogénea entre el eslabón 5 y la base del robot desde el árbol TF (`tf2_ros`).

Los vectores de rotación y traslación recopilados se procesaron mediante la función `cv2.calibrateHandEye` especificando el método de cuaterniones duales de Daniilidis (`cv2.CALIB_HAND_EYE_DANIILIDIS`). La matriz de transformación homogénea resultante $T_{ee \rightarrow cam}$ de 4×4 permitió proyectar con precisión las coordenadas tridimensionales capturadas por la cámara hacia el sistema de referencia global del brazo robótico, exportando los resultados en formatos `.yaml` y `.npz` para su integración en el resto del sistema.

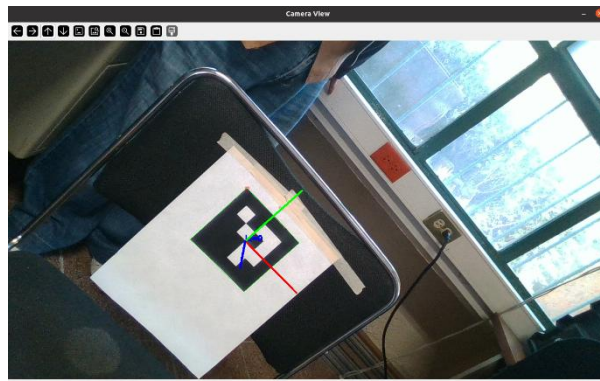


Figura 2. Vista del ArUco en posición correcta.

El nodo `HandEyeCalibration` se suscribe a la imagen de color de la cámara y, cada vez que el usuario confirma una nueva pose del brazo (presionando `Enter`), captura simultáneamente:

- a) La transformación del efector final respecto a la base, leída del árbol TF.
- b) La pose del marcador ArUco respecto a la cámara, estimada mediante `estimatePoseSingleMarkers`.

Tras acumular 20 pares de poses, se invoca `cv2.calibrateHandEye` con el método `CALIB_HAND_EYE_DANIILIDIS` para resolver la transformación cámara-efector final.

6.2. Segmentación

Uno de los principales retos del sistema fue identificar, mediante un módulo de visión, frutas en una primera etapa y, posteriormente, cubos de fruta en fases más avanzadas del proyecto. Para lograrlo se implementaron dos etapas, debido a las limitaciones de procesamiento del equipo, las cuales serían solucionadas en fases posteriores.

6.2.1. Segmentación híbrida mediante YOLOv8s y SAM2_b

El objetivo principal de esta primera etapa del módulo de visión es detectar, segmentar y ubicar en 3D los objetos de interés para que el brazo robótico pueda ejecutar la manipulación y el agarre en tareas de alimentación asistida. Dado que utilizar únicamente cuadros delimitadores (*bounding boxes*) no proporciona el contorno exacto del objeto, complicando la planeación de la trayectoria, se implementó una estrategia en dos etapas que combina la rapidez de YOLOv8 con la precisión de SAM 2.

Para esta fase del sistema, la arquitectura se simplificó a un esquema operativo compuesto por únicamente dos nodos principales en ROS, un nodo de segmentación y un nodo de transformación espacial. El nodo de segmentación ejecuta una versión beta que integra la variante ligera YOLOv8s (*small*) para la detección primaria y la versión base de SAM 2 (*Segment Anything Model 2*) para la extracción fina del área de interés.

En este flujo, YOLOv8s procesa la imagen en tiempo real e identifica un conjunto de 16 clases específicas del dataset COCO, enfocadas en personas, utensilios y alimentos:

- **Usuario:** *Person* (ID 0).
- **Utensilios y recipientes:** *Cup* (ID 41), *Fork* (ID 42), *Knife* (ID 43), *Spoon* (ID 44) y *Bowl* (ID 45).
- **Alimentos:** *Banana* (ID 46), *Apple* (ID 47), *Sandwich* (ID 48), *Orange* (ID 49), *Broccoli* (ID 50), *Carrot* (ID 51), *Hot dog* (ID 52), *Pizza* (ID 53), *Donut* (ID 54) y *Cake* (ID 55).

Una vez que YOLOv8s reconoce alguna de estas clases y genera la caja delimitadora, esta región sirve como *prompt* espacial para que el modelo SAM 2 genere la máscara de segmentación precisa a nivel de píxel, aislando el contorno real del objeto y descartando el fondo.

A partir de la máscara binaria resultante, el nodo de segmentación calcula el centro de masa 2D (u_c, v_c) en el plano de la imagen mediante sus momentos. Utilizando la información de profundidad alineada proveniente de la cámara Intel RealSense D415 y sus parámetros intrínsecos, esta coordenada se proyecta al espacio tridimensional (X_c, Y_c, Z_c) en el sistema de referencia propio de la cámara (*camera_link*) (Bradski, 2000), mediante las ecuaciones

$$X_c = \frac{(u_c - c_x) \cdot Z_c}{f_x} \text{ y } Y_c = \frac{(v_c - c_y) \cdot Z_c}{f_y}.$$

Este nodo publica la posición 3D estimada en el tópico */object_centroid* (con tipo de mensaje *geometry_msgs/PointStamped*) y transmite la nube de puntos filtrada en */object_pointcloud*, dejando estos datos disponibles en la red de ROS para que posteriormente el nodo de transformación realice la conversión al marco de referencia de la base del robot (*m1n6s300_link_base*). Todo el desarrollo del módulo de visión se integró en ROS Noetic

con librerías como OpenCV, PyRealSense2, Ultralytics y PyTorch, validando su correcto funcionamiento y la precisión del centroide 3D en el simulador Gazebo previo a las pruebas en hardware real.

Cabe destacar que el funcionamiento de este nodo híbrido se emplea únicamente para simulación, ya que a la altura del desarrollo de este código no se contaba con los recursos necesarios para ejecutar un modelo con mayores requerimientos para su procesamiento como SAM 3. Esta simulación la podemos observar mediante Gazebo.

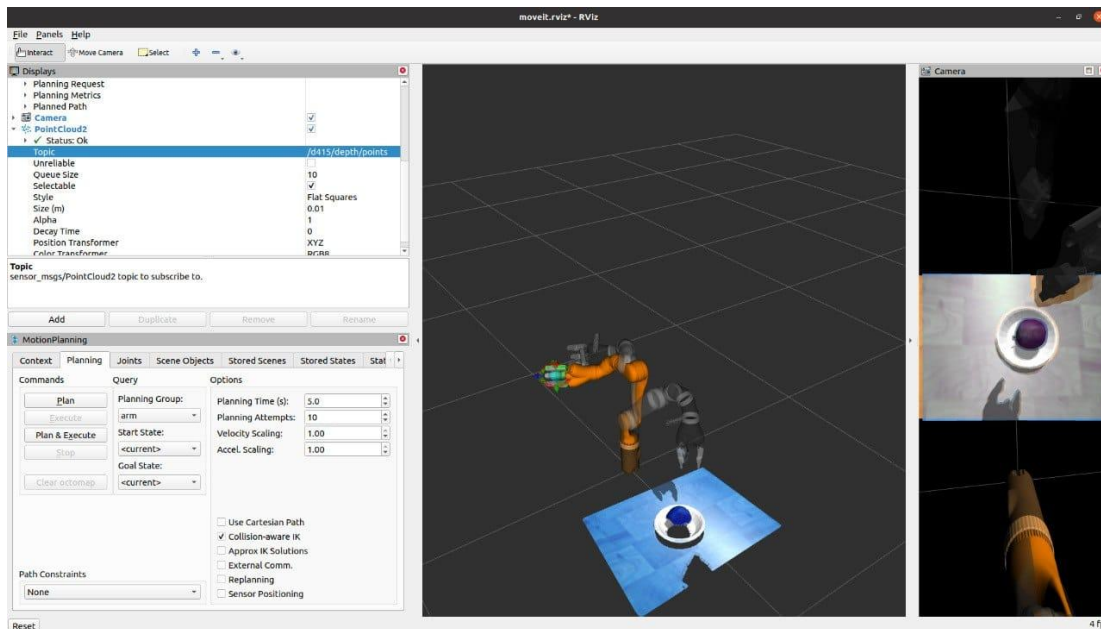


Figura 3. Pantalla de Rviz con simulación mediante segmentación híbrida.

Una de las principales limitaciones de este planteamiento híbrido es que se restringe a las clases predefinidas por el modelo YOLOv8s. Sin embargo, el funcionamiento del brazo robótico requiere trabajar con fruta picada, una categoría que no se encuentra entre las clases reconocidas por el modelo.

6.2.2. Segmentación mediante SAM 3 (*Segment Anything with Concepts*)

Durante la segunda etapa del desarrollo del módulo de segmentación se escaló hacia un modelo más potente basado en el uso de conceptos. Uno de los principales retos fue la segmentación de los cubos de fruta, ya que presentan diversas dificultades: formas irregulares, tendencia a agruparse y un elevado nivel de ruido durante el proceso de segmentación. Todos estos aspectos fueron verificados mediante pruebas realizadas en la primera etapa del desarrollo.

6.2.2.1. Arquitectura general del nodo

Con la disponibilidad de la infraestructura de cómputo adecuada para el procesamiento de modelos fundacionales en tiempo real, la arquitectura del módulo de visión evolucionó hacia un esquema de segmentación semántica directa basada en prompts utilizando SAM 3

(SAM3SemanticPredictor). A diferencia de las aproximaciones convencionales en dos etapas que dependen de un detector de objetos primario (como YOLOv8) para generar cajas delimitadoras previas, esta implementación prescinde del detector primario y aprovecha la capacidad de SAM 3 para interpretar prompts de texto y aislar objetos de interés directamente en la escena.

El procesamiento completo está encapsulado dentro del nodo en ROS denominado vision_d415_sam3 (KinovaVisionSAM3). Este nodo integra la ingesta de imágenes RGB e información de profundidad de la cámara Intel RealSense D415, ejecuta la inferencia del modelo SAM 3, aplica un filtrado geométrico estricto para la discriminación de piezas individuales y publica la ubicación tridimensional del objetivo mediante mensajes estandarizados de ROS.

6.2.2.2. Filtrado geométrico y discriminación de máscaras (MaskFilter)

La inferencia directa de SAM 3 a partir de un prompt conceptual como *"a small cube of fruit"* suele generar múltiples máscaras ambiguas o jerárquicas que incluyen desde ruido de alta frecuencia hasta la fruta completa o el recipiente que la contiene. El modelo consta de la siguiente configuración:

Nombre del parámetro	Descripción	Valor
conf	Umbral mínimo de confianza (Confidence Threshold) para filtrar las detecciones/segmentaciones válidas.	0.2
task	Define el tipo de tarea que realizará el modelo (en este caso, segmentación de objetos).	"segment"
mode	Especifica la modalidad de ejecución del predictor (inferencia/predicción sobre los datos de entrada).	"predict"
model	Ruta relativa o absoluta del archivo de pesos entrenados (weights) del modelo a cargar.	os.path.join(package_path, "weights", "sam3.pt")
iou	Umbral de intersección sobre unión (Intersection over Union) utilizado para la supresión de no máximos (NMS) y evitar detecciones duplicadas.	0.3
half	Habilita el uso de precisión media (FP16 / Float16) para	True

	acelerar la inferencia en GPU y reducir el uso de memoria VRAM.	
--	---	--

Figura 4. Configuración del modelo de SAM 3.

Para garantizar la selección exclusiva de cubos de fruta individuales sin manipular la estructura completa ni pequeños artefactos visuales, el nodo implementa una clase especializada de post-procesamiento denominada MaskFilter.

La limpieza de las máscaras binarias crudas generadas por el predictor se ejecuta en dos fases secuenciales:

Filtros básicos de escala, proporción, compacidad y contención jerárquica

En la primera fase de limpieza, ejecutada por el método `_basic_filter`, la clase MaskFilter evalúa secuencialmente cada una de las máscaras binarias candidatas generadas por SAM 3. Esta evaluación se realiza de forma individual a partir de la matriz booleana de cada máscara, aplicando tres criterios matemáticos y geométricos consecutivos para garantizar que solo las regiones que representan cubos individuales válidos avancen a las siguientes etapas de procesamiento.

1. Filtro de área mínima (Escala absoluta)

El filtro de área mínima actúa como una primera barrera contra la alta sensibilidad que caracteriza a los modelos fundacionales de segmentación. Durante la inferencia, es común que se generen pequeñas regiones de segmentación espurias compuestas por apenas un puñado de píxeles, provocadas por ruido de alta frecuencia en el sensor óptico, pequeñas sombras o reflexiones especulares en la superficie de la fruta.

Para erradicar estos artefactos, el algoritmo calcula el área absoluta de la máscara sumando el número total de píxeles activos ($A_{\text{máscara}} = \sum \text{mask_bool}$). Si este valor es inferior al umbral `min_area` (configurado en 100 píxeles), la máscara se descarta inmediatamente mediante la condición `mask_area < self.min_area`. Esto evita procesar regiones irrelevantes y reduce significativamente el costo computacional de las etapas posteriores.

2. Filtro de área máxima relativa (Escala proporcional)

A diferencia del filtro absoluto, el filtro de área máxima evalúa la escala del objeto en relación con la resolución global del fotograma. SAM 3, al recibir un *prompt* conceptual, puede segmentar accidentalmente el plato completo, el contenedor o la masa completa de frutas en lugar de los cubos individuales.

El algoritmo obtiene el área total del fotograma ($A_{\text{imagen}} = \text{alto} \times \text{ancho}$) y calcula la fracción ocupada por la máscara. Si la proporción $A_{\text{máscara}}/A_{\text{imagen}}$ supera el valor asignado a `max_area_ratio` (0.04, equivalente al 4% del área total de la imagen), la máscara se rechaza automáticamente mediante la instrucción `mask_area / img_area > self.max_area_ratio`. Este

umbral garantiza que cualquier estructura que rebase las dimensiones físicas esperadas de un único cubo de fruta sea descartada de la selección.

3. Filtro de compacidad y forma geométrica (Relación máscara-Bounding Box)

Incluso si una máscara cumple con los límites de área mínima y máxima, su morfología puede no corresponder a la de un cubo de fruta. Las segmentaciones alargadas, los bordes deshilachados o las regiones discontinuas suelen ser indicativos de una segmentación defectuosa o de sombras proyectadas.

Para validar la solidez geométrica de la región, el método extrae los límites espaciales extremos de la máscara sobre los ejes de la imagen mediante $ys, xs = np.where(mask_bool)$. A partir de estos índices, calcula el área del rectángulo delimitador alineado con los ejes (*bounding box* no orientado):

$$A_{\text{bbox}} = (x_{\text{máx}} - x_{\text{mín}} + 1) \times (y_{\text{máx}} - y_{\text{mín}} + 1)$$

Posteriormente, evalúa la razón de compacidad C , definida como el cociente entre el área real ocupada por la máscara y el área de su *bounding box*:

$$C = \frac{A_{\text{máscara}}}{A_{\text{bbox}} + \epsilon}$$

Donde $\epsilon = 1 \times 10^{-6}$ se añade para prevenir errores de división por cero. Dado que un objeto compacto y con bordes definidos (como un cubo proyectado en 2D) ocupa una porción sustancial de su cuadro delimitador, el algoritmo exige que esta relación sea igual o superior al parámetro `min_compactness` (0.10). Si $C < 0.10$, la máscara es rechazada por presentar una forma excesivamente dispersa o irregular.

Únicamente aquellas máscaras que superan con éxito los tres controles secuenciales (área mínima, área máxima relativa y compacidad) son integradas en las listas `kept_masks` y `kept_indices`, quedando indexadas y preparadas para ingresar a la segunda fase de filtrado jerárquico por contención (`_remove_containing_masks`).

4. Filtro de proporción mínima de tamaño para estructuras contenedoras (size_ratio_min)

Una vez superados los filtros individuales, las máscaras aprobadas pasan al método `_remove_containing_masks`, el cual analiza las relaciones geométricas entre pares de máscaras candidatas para resolver la jerarquía espacial entre objetos. En escenarios de fruta picada, SAM 3 tiende a segmentar tanto el cubo individual como agrupaciones más grandes de piezas adyacentes.

Para evitar la supresión accidental entre dos cubos adyacentes de dimensiones similares, el algoritmo exige una diferencia de escala considerable antes de considerar que una máscara j pueda ser la "contenedora" de una máscara i . Mediante el aplanado de matrices y operaciones vectorizadas, calcula la razón de área entre cada par de máscaras ($S_{i,j} = A_j/A_i$). El sistema

establece que la máscara j debe poseer un área al menos tres veces mayor que la máscara i , según la condición $\text{size_ratio} \geq \text{self.size_ratio_min}$ (donde $\text{size_ratio_min} = 3.0$). Si no se satisface esta diferencia de magnitud, la máscara j no se clasifica como candidata a entidad contenedora, protegiendo así la integridad de cubos individuales de tamaños semejantes.

5. Filtro de umbral de contención Inter-Máscara (containment_thresh)

Complementando el criterio de tamaño, este quinto parámetro evalúa el grado de traslape e inclusión espacial entre las regiones segmentadas. El algoritmo calcula la matriz de intersección entre las representaciones booleanas aplanadas de todas las máscaras y determina la proporción de área de la máscara pequeña i que reside estrictamente dentro de los límites de la máscara grande j :

$$\text{Contención}_{i,j} = \frac{A_i \cap A_j}{A_i + 1 \times 10^{-6}}$$

El parámetro `containment_thresh` se configura en un valor estricto del 0.92 (92%), evaluado mediante la instrucción `containment >= self.containment_thresh`. Una máscara grande j únicamente se marca para eliminación (`to_remove`) si encierra en su interior un porcentaje igual o superior al 92% de la superficie de la máscara pequeña i . Un umbral elevado de contención es crítico en tareas de manipulación robótica, ya que evita eliminar máscaras de cubos que simplemente se encuentran en contacto o ligeramente solapados por la perspectiva de la cámara, asegurando la eliminación exclusiva de máscaras envolventes de alta jerarquía (padres) y conservando únicamente los cubos individuales verdaderos (hijos).

A partir del uso de estos filtros, podemos obtener un cubo único sin ruido ni traslapes.

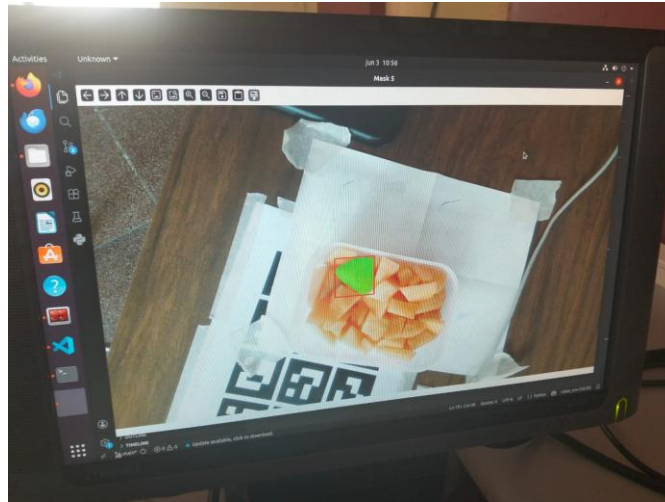


Figura 5. Segmentación a partir de los filtros de MaskFilter.

6.2.2.3. *Preprocesamiento de imagen y normalización fotométrica*

Las variaciones en las condiciones de iluminación ambiental, las sombras proyectadas por la estructura del propio robot y los reflejos especulares sobre la superficie húmeda de los trozos

de fruta representan desafíos significativos para la consistencia en la segmentación semántica. Para mitigar estos efectos visuales y maximizar la precisión de las máscaras generadas por SAM 3, el nodo ejecuta un preprocesamiento fotométrico en la función `_normalize_image`.

Es importante destacar que el fotograma original en formato BGR procedente del tópico `/d415/color/image_raw` no sufre ninguna alteración destructiva. Este se conserva en su estado normal para las tareas de renderizado, trazado de contornos y visualización gráfica en interfaz. En su lugar, se opera sobre una copia orientada exclusivamente a la inferencia del modelo.

Transformación al espacio de color CIE LAB

El primer paso del proceso consiste en convertir la imagen BGR original al espacio de color fotométrico CIE LAB mediante la función `cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2LAB)`. A diferencia de los espacios RGB o BGR (donde la información de luminancia y cromaticidad se encuentra acoplada en los tres canales de color), el espacio LAB desacopla de manera efectiva la intensidad luminosa de la percepción del color:

- Canal *L*: Representa la luminancia (brillo relativo), variando desde el negro absoluto hasta el blanco.
- Canal *A*: Modula la cromaticidad en el eje verde-rojo.
- Canal *B*: Modula la cromaticidad en el eje azul-amarillo.

Al aislar el canal *L* con la función `cv2.split(lab)`, el sistema puede manipular directamente el contraste y la distribución luminosa de la escena sin distorsionar la tonalidad natural o la saturación cromática de los trozos de fruta.

Ecualización adaptativa del histograma (CLAHE)

Sobre el canal de luminancia *L* extraído, se aplica el algoritmo de ecualización adaptativa de histograma con límite de contraste (CLAHE, por sus siglas en inglés *Contrast Limited Adaptive Histogram Equalization*).

La ecualización tradicional de histogramas globales suele sobresaturar regiones con mucha luz o amplificar el ruido en zonas oscuras. CLAHE resuelve este inconveniente dividiendo la imagen en una cuadrícula no superpuesta de regiones locales denominadas tiles (configuradas en un tamaño de 8×8 píxeles) y calculando un histograma independiente para cada sección.

El parámetro `clipLimit = 2.0` establece un umbral de recorte rígido sobre las crestas del histograma local. Cualquier píxel que supere dicho límite de contraste es redistribuido de manera uniforme entre todos los contenedores (bins) del histograma antes de aplicar la función de distribución acumulativa. Esto evita el incremento excesivo del ruido en zonas homogéneas mientras resalta el contraste en las fronteras entre cubos adyacentes,

diferenciando con claridad las sombras delgadas que separan a dos piezas de fruta en estrecho contacto.

Reconstrucción y almacenamiento temporal para inferencia

Una vez ecualizado el canal de luminancia (L_{eq}), los tres canales se recombinan en el espacio LAB y se reconvierten de vuelta al formato BGR. Dado que el predictor de SAM 3 requiere la lectura del fotograma desde el almacenamiento secundario, la imagen normalizada resultante se escribe en un directorio temporal aislado generado dinámicamente (`tempfile.mkdtemp(prefix="sam3_ros_")`) bajo el nombre de archivo `current_frame.jpg`. Posteriormente, la ruta absoluta de este archivo es dada al método `self.predictor.set_image(self._tmp_frame_path)` para la posterior ejecución de la segmentación orientada por prompts de texto.

6.2.2.4. Criterio adaptativo de selección del cubo objetivo

Una vez que el módulo de filtrado geométrico entrega un conjunto de máscaras binarias limpias y etiquetadas, el sistema debe tomar una decisión autónoma para determinar cuál de todos los cubos detectados constituye el objetivo óptimo para la maniobra de sujeción del brazo robótico. Esta decisión es vital, ya que seleccionar una pieza ubicada en los bordes del recipiente o atrapada bajo otras capas puede provocar colisiones con la herramienta terminal o fallos en el agarre.

Para resolver este problema, la función `_select_target` implementa un algoritmo de selección adaptativo que evalúa la distribución espacial del grupo de frutas en el plano 2D de la imagen y la combina con la información de profundidad tridimensional (Z). El comportamiento del algoritmo se ajusta dinámicamente en función de la densidad de piezas presentes en la escena.

Extracción de centroides

En primera instancia, el algoritmo determina la posición 2D de cada cubo candidato en el espacio de la imagen. Para cada máscara booleana k , la matriz de píxeles se convierte a un formato de imagen de 8 bits. Sobre esta representación, se calculan los momentos geométricos de la imagen mediante la función `cv2.moments(mask_uint8)`:

- Momento de orden cero (M_{00}): Representa la superficie o área total acumulada de la máscara en píxeles activos. Si $M_{00} = 0$ (un caso extremo de degeneración de la máscara), se asigna el centro óptico de la imagen como medida de contingencia para evitar errores por división entre cero.
- Momentos de primer orden (M_{10} y M_{01}): Representan la suma ponderada de las coordenadas de los píxeles a lo largo de los ejes X e Y , respectivamente.

A partir de estos momentos, se derivan las coordenadas del centroide (u_k, v_k) en el plano de la imagen original ajustando por la escala entre la máscara y el fotograma de la cámara:

$$u_k = \frac{M_{10}}{M_{00}} \cdot \frac{\text{ancho}_{\text{imagen}}}{\text{ancho}_{\text{máscara}}}, \quad v_k = \frac{M_{01}}{M_{00}} \cdot \frac{\text{alto}_{\text{imagen}}}{\text{alto}_{\text{máscara}}}$$

Una vez obtenidos los centroides de todas las N máscaras candidatas, se calcula el centro de masa global del grupo de frutas $(\bar{u}, \bar{v})$ promediando sus posiciones espaciales:

$$(\bar{u}, \bar{v}) = \frac{1}{N} \sum_{k=1}^N (u_k, v_k)$$

Posteriormente, se determina la distancia euclidiana desde el centroide de cada cubo k hacia el centro geométrico del grupo:

$$d_k = \sqrt{(u_k - \bar{u})^2 + (v_k - \bar{v})^2}$$

Cálculo del centro del grupo

Para seleccionar uno de los cubos candidatos en la ejecución del movimiento del brazo, se consideran criterios específicos orientados a evitar colisiones con el contenedor.

Modalidad normal u operación en grupo ($N > \text{FEW_THRESHOLD}$)

Cuando la escena contiene una densidad elevada de piezas (definida por la condición $N > 4$, según el umbral $\text{FEW_THRESHOLD} = 4$), la estrategia prioriza vaciar el plato desde el centro hacia la periferia. Tomar piezas ubicadas en los márgenes exteriores presenta un riesgo elevado de que el efector final choque contra las paredes del contenedor.

Para mitigar esto, el algoritmo ordena los índices de las máscaras de menor a mayor distancia d_k respecto al centro del grupo y extrae el subconjunto equivalente al 30% de los cubos más céntricos ($\text{CENTER_FRACTION} = 0.30$).

De entre este grupo restringido de cubos céntricos, el sistema evalúa la profundidad tridimensional representativa Z asociada a cada uno y selecciona formalmente como objetivo aquel cubo que posea el menor valor de profundidad (es decir, la pieza situada a mayor altura o en el tope del apilamiento central).



Figura 6. Selección de cubo de fruta picada en modo normal.

Modalidad final o despacho de piezas restantes ($N \leq \text{FEW_THRESHOLD}$)

Conforme el brazo remueve la fruta y quedan cuatro o menos piezas en la escena ($N \leq 4$), el agrupamiento espacial pierde significancia y la noción de un "centro del grupo" deja de ser representativa.

En esta fase, la función cambia automáticamente a la modalidad final. El criterio de centrado en 2D se inhabilita completamente y el sistema evalúa a todos los cubos restantes directamente en el dominio 3D. El algoritmo descarta cualquier región sin lecturas de profundidad válidas ($Z = \infty$) y selecciona de forma directa la pieza que presente la menor profundidad Z_m . Esta transición fluida asegura que, hacia el término de la tarea, cualquier cubo accesible en la mesa pueda ser tomado de inmediato sin ser rechazado arbitrariamente por restricciones de proximidad al centro.

6.2.2.5. Mapeo de profundidad y proyección 3D

Para trasponer las detecciones bidimensionales obtenidas en la imagen hacia el espacio cartesiano tridimensional accesible por el manipulador robótico, el sistema integra la información fotométrica con la nube de puntos proveniente del sensor de profundidad.

Filtrado espacial de la nube de puntos

El procedimiento inicia con el filtrado de la nube de puntos almacenada en la variable `self.last_cloud`, descartando las lecturas nulas o aquellas cuya profundidad resulte no válida ($Z \leq 0$).

Proyección perspectiva inversa de puntos 3D

Cada punto tridimensional $P = (X, Y, Z)^T$ se proyecta sobre el plano focal de la cámara utilizando el modelo de cámara estenopeica (*pinhole*). Mediante los parámetros intrínsecos del sensor (f_x, f_y, c_x, c_y), se determinan sus coordenadas equivalentes en píxeles (u, v) dentro de la imagen original:

$$u = \left\lfloor \frac{X \cdot f_x}{Z} + c_x \right\rfloor, \quad v = \left\lfloor \frac{Y \cdot f_y}{Z} + c_y \right\rfloor$$

Los puntos cuya proyección sobrepasa los límites físicos de la imagen ($0 \leq u < \text{ancho}_{\text{imagen}}$ y $0 \leq v < \text{alto}_{\text{imagen}}$) son eliminados del cálculo.

Mapeo y muestreo sobre la máscara binaria

Dado que la máscara binaria resultante del modelo de segmentación puede presentar dimensiones distintas a las del fotograma original, las coordenadas proyectadas se reescalan proporcionalmente hacia el espacio discreto de la máscara ($u_{\text{mask}}, v_{\text{mask}}$):

$$u_{\text{mask}} = \left\lfloor u \cdot \frac{\text{ancho}_{\text{máscara}}}{\text{ancho}_{\text{imagen}}} \right\rfloor, \quad v_{\text{mask}} = \left\lfloor v \cdot \frac{\text{alto}_{\text{máscara}}}{\text{alto}_{\text{imagen}}} \right\rfloor$$

Evaluando la matriz booleana correspondiente, el algoritmo conserva únicamente aquellos puntos 3D cuya proyección coincide con la región activa de la fruta.

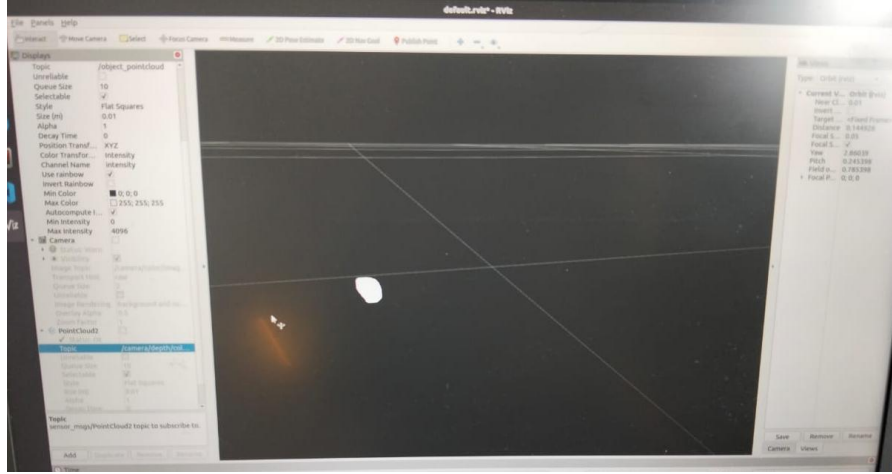


Figura 7. Proyección de la máscara sobre nube de puntos

Estimación de profundidad representativa y coordenadas 3D

A partir del conjunto de lecturas de profundidad válidas $\{Z_1, Z_2, \dots, Z_m\}$, se obtiene la profundidad representativa Z_m de la pieza calculando su media aritmética:

$$Z_m = \frac{1}{m} \sum_{i=1}^m Z_i$$

En ausencia de puntos 3D dentro de la máscara, la profundidad se define como infinita, descartando el candidato. Finalmente, para el cubo objetivo seleccionado, las coordenadas tridimensionales de su centroide en el sistema de referencia de la cámara (X_c, Y_c, Z_m) se

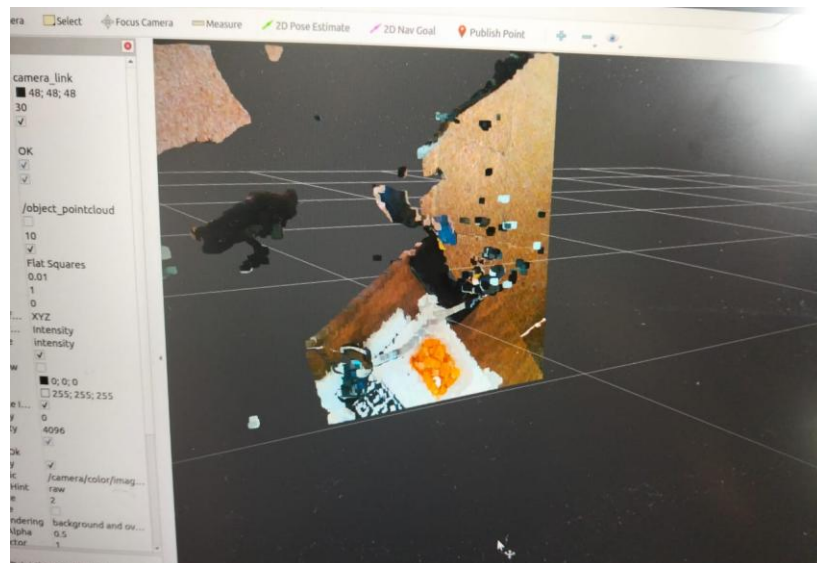


Figura 8. Visualización de máscara proyectada en nube de puntos

reconstruyen algebraicamente a partir de sus coordenadas en píxeles $(u_{\text{target}}, v_{\text{target}})$ mediante las expresiones:

$$X_c = \frac{(u_{\text{target}} - c_x) \cdot Z_m}{f_x}, \quad Y_c = \frac{(v_{\text{target}} - c_y) \cdot Z_m}{f_y}$$

6.2.2.6. Publicación de los resultados en tópicos de ROS

Una vez determinado el cubo objetivo y calculada su posición en el espacio tridimensional, el nodo transmite la información a la arquitectura de ROS.

Publicación del centroide objetivo

La posición cartesiana del objetivo se emite en el tópico `object_centroid` mediante la función `_publish_centroid`, utilizando una estructura de mensaje de tipo `geometry_msgs/PointStamped`. La estampa temporal se genera con el tiempo actual del sistema (`rospy.Time.now()`), y el marco de referencia (`frame_id`) se asigna directamente al sistema de coordenadas de la cámara (`self.cam_frame`). Esto permite que los nodos subsiguientes de planificación de trayectorias realicen la transformación homogénea hacia la base del brazo robótico (`m10s300_link_base`).

Publicación de la nube de puntos segmentada

De forma paralela, a través del publicador `pc_pub`, el sistema construye una sub-nube de puntos correspondiente de manera exclusiva al objeto seleccionado con fines de diagnóstico e inspección en RViz. La estructura completa se empaqueta como un mensaje de tipo `sensor_msgs/PointCloud2` con campos de posición (x, y, z), el cual es enviado al tópico `/object_pointcloud`. Esta estrategia de publicación proporciona los insumos necesarios tanto para la ejecución táctica del agarre como para la supervisión remota del proceso de segmentación.

6.3. Transformación de coordenadas al espacio del robot

El último paso dentro de la cadena de procesamiento de visión consiste en convertir los centroides de los objetos detectados (referenciados inicialmente en el sistema de coordenadas de la cámara) al espacio cartesiano global de la base del robot. Con este fin, se desarrolló e implementó el nodo dedicado de ROS denominado `direct_transformer`, el cual automatiza y ejecuta este cálculo de conversión espacial en tiempo real dado los tópicos de segmentación.

6.3.1. Configuración ojo en mano (*Hand-in-Eye*)

Dado que el sensor de visión se encuentra montado directamente sobre el último eslabón del brazo robótico, el sistema opera bajo una configuración *Hand-in-Eye*. En esta arquitectura, el sensor se desplaza solidariamente con la estructura cinemática durante las maniobras de exploración y aproximación. La relación geométrica estática entre el centro óptico de la cámara y la estructura física del efecto final (eslabón 5) fue determinada previamente

mediante un proceso de calibración mano-ojo, resultando en una matriz de transformación homogénea fija $T_{\text{cam}}^{\text{link}_5}$.

6.3.2. Formulación matemática y árbol de transformaciones (TF)

El nodo `direct_transformer` recibe los centroides bidimensionales en píxeles generados por la etapa de segmentación, extrae su profundidad representativa Z mediante la nube de puntos y proyecta la coordenada en el espacio homogéneo 3D de la cámara $P_{\text{cam}} = (X_c, Y_c, Z_c, 1)^T$.

Para trasladar esta posición al origen global del manipulador, el nodo consulta dinámicamente el árbol de transformaciones de ROS (tf), leyendo en tiempo real la postura del robot proporcionada por los encoders. La relación espacial instantánea para proyectar la coordenada del objeto hacia la base global del brazo Kinova (`m1n6s300_link_base`) se expresa algebraicamente como:

$$P_{\text{robot}} = T_{\text{link}_5}^{\text{base}} \cdot T_{\text{cam}}^{\text{link}_5} \cdot P_{\text{cam}}$$

Donde:

- $P_{\text{cam}} = (X_c, Y_c, Z_c, 1)^T$ representa la coordenada homogénea 3D del centroide calculada respecto al sistema de referencia de la cámara.
- $T_{\text{cam}}^{\text{link}_5}$ corresponde a la matriz estática de transformación homogénea 4×4 obtenida mediante la calibración mano-ojo, la cual acopla la pose del sensor respecto al eslabón 5 del robot.
- $T_{\text{link}_5}^{\text{base}}$ representa la transformación cinemática dinámica desde el eslabón 5 (efector final) hasta la base global Kinova (`m1n6s300_link_base`), calculada en tiempo real a partir de la representación en cuaterniones de la orientación y el vector de traslación del brazo.
- $P_{\text{robot}} = (X_r, Y_r, Z_r, 1)^T$ es la coordenada cartesiana final del objeto referenciada de forma absoluta al marco de origen del brazo robótico.

6.3.3. Implementación algorítmica en ROS

A nivel de software, el cálculo cinemático y la interpolación temporal se resuelven mediante un escuchador de transformaciones de la librería tf. El nodo suscribe la información del centroide en el marco de la cámara y emite la posición corregida hacia la base del brazo robótico mediante el siguiente script de procesamiento, ver Anexo 1.

6.3.4. Sincronización temporal y publicación para la planeación de agarre

La utilización de los métodos `waitForTransform` e `transformPoint` en el nodo garantiza una estricta coherencia temporal entre la captura del fotograma y la postura física del brazo, previniendo desfases geométicos derivados del movimiento continuo del sistema.

Una vez obtenido el punto cartesiano P_{robot} , la coordenada tridimensional resultante se emite de forma continua en el t3pico /object_centroid_robot mediante un mensaje de tipo geometry_msgs/PointStamped. Esta publicaci3n desacopla la percepci3n visual del movimiento del robot y proporciona una ubicaci3n cartesiana invariante ante desplazamientos de la c3mara, sirviendo como el insumo directo para la m3quina de estados y los algoritmos de cinem3tica inversa encargados de la planeaci3n y ejecuci3n de la trayectoria de agarre.

7. Resultados obtenidos

Al concluir el periodo de pr3cticas profesionales y finalizar las pruebas de integraci3n en el entorno de simulaci3n (Gazebo) y en la plataforma f3sica del manipulador Kinova Gen2, se valid3 el desempe1o t3cnico del sistema de visi3n por computadora y transformaci3n espacial.

7.1. Desempe1o del m3dulo de percepci3n y segmentaci3n

La incorporaci3n del preprocesamiento fotom3trico mediante el espacio de color LAB y la t3cnica CLAHE en el canal de luminancia (L) estabiliz3 la entrada crom3tica hacia el modelo SAM 3, reduciendo la sensibilidad a las fluctuaciones de luz en el laboratorio.

Por su parte, la implementaci3n de la clase MaskFilter resolvi3 las inconsistencias de sobre-segmentaci3n y falsos positivos mediante la aplicaci3n rigurosa de cuatro criterios geom3tricos:

- 3rea m3nima: Eliminaci3n de ruido de alta frecuencia (< 100 px).
- L3mite de 3rea m3xima: Exclusi3n del contorno del recipiente o fondo ($> 4\%$ del 3rea total).
- Compacidad: Descarte de m3scaras irregulares o no conexas ($< 10\%$ de relaci3n 3rea/caja delimitadora).
- Solapamiento y contenci3n: Filtrado de m3scaras agrupadoras ($> 92\%$ de intersecci3n), conservando exclusivamente cubos de fruta individuales.

7.2. Validaci3n de la l3gica de selecci3n y transformaci3n espacial

El algoritmo de selecci3n adaptativa evalu3 de forma efectiva los escenarios de densidad de alimento dentro del contenedor. Durante el *Modo Normal* ($N > 4$), la restricci3n del 30% central combinada con la menor coordenada de profundidad (Z) evit3 colisiones de la pinza con las paredes del bowl. En el *Modo Final* ($N \leq 4$), la desactivaci3n del filtro de centralidad permiti3 recolectar piezas aisladas sin p3rdidas de detecci3n.

En la etapa de alineaci3n espacial, la sincronizaci3n de los t3picos de imagen RGB, la matriz de par3metros intr3secos del sensor Intel RealSense D415 y la nube de puntos (PointCloud2)

permitió proyectar los centroides en espacio 3D. El nodo `direct_transformer` procesó y transformó estas coordenadas en tiempo real hacia el marco de referencia de la base del robot (`m1n6s300_link_base`) a una frecuencia superior a 15 Hz.

7.3. Integración con el sistema control y guiado

La canalización de los datos de posición tridimensional desde el módulo de visión hacia la máquina de estados y el planificador de trayectorias (desarrollados en colaboración con el equipo) permitió completar la rutina de alimentación asistida de forma secuencial, coordinando de manera continua las etapas de aproximación, descenso, sujeción y retorno del manipulador.

7.4. Oportunidades de Mejora y Trabajo Futuro

A partir de la evaluación experimental del prototipo, se establecen las siguientes líneas de optimización para futuras iteraciones del proyecto:

- Optimizar los tiempos de inferencia del modelo SAM 3 mediante la conversión de pesos a formatos de ejecución cuantitativa (e.g., TensorRT o ONNX Runtime), aumentando la tasa de refresco global del sistema.
- Implementar un filtro de estimación de estados (como un filtro de Kalman o media móvil exponencial) sobre las coordenadas de la nube de puntos para mitigar variaciones en la profundidad causadas por ruidos especulares en la superficie de los alimentos o del recipiente.
- Ampliar los parámetros de la clase `MaskFilter` para permitir la identificación adaptativa de alimentos con geometrías no cúbicas o dispuestos en recipientes con formas complejas.

8. Conclusiones

La culminación de estas prácticas profesionales en el Área de Electrónica del INAOE representa un hito crucial para comprender la complejidad técnica asociada a llevar la teoría de la ingeniería en ciencias de la computación hacia entornos físicos reales. El desarrollo de este proyecto demostró que el verdadero desafío de la ingeniería no radica únicamente en programar de forma aislada un algoritmo avanzado de segmentación como SAM 3 o en resolver las ecuaciones formales de transformaciones espaciales en papel. El reto principal consiste en lograr que todo este flujo de percepción tridimensional y procesamiento de datos responda con alta precisión, baja latencia y de manera óptima ante las restricciones físicas de tiempo real y de hardware que imponen los sistemas embebidos complejos.

Por otra parte, la experiencia adquirida consolida la premisa de que los desarrollos de software, visión artificial y automatización no ocurren en aislamiento. El éxito integral de la plataforma no se habría alcanzado sin una articulación interdisciplinaria y colaborativa; la

acoplación directa entre el pipeline de visión por computadora desarrollado, encargado del preprocesamiento, la segmentación mediante MaskFilter y la transformación de coordenadas, con el módulo de control de trayectorias físicas y la máquina de estados secuencial (desarrollados conjuntamente con una compañera de prácticas) fue el eslabón indispensable para la integración del sistema. Esta sinergia permitió transformar las predicciones abstractas de píxeles y nubes de puntos en un comportamiento robótico coordinado, útil, seguro y con aplicación directa en tareas asistenciales.

Asimismo, los resultados obtenidos aportan un valor metodológico significativo al campo de la robótica. Se establecieron metodologías rigurosas y reproducibles para la calibración mano-ojo mediante marcadores ArUco, el procesamiento directo de imágenes con modelos fundacionales sin detectores intermedios y el mapeo tridimensional en tiempo real a través de ROS. Estos procedimientos no solo validan experimentalmente la factibilidad técnica de este sistema de alimentación asistida mediante el manipulador Kinova Gen2, sino que asientan bases firmes, metodologías transferibles y rutas claras de trabajo futuro para el desarrollo de proyectos avanzados dentro del área de la robótica asistencial y la interacción humano-robot.

Referencias

- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 25(11), 120–125.
- Carfagni, M., Furferi, L., Furferi, R., Puggelli, L., & Volpe, Y. (2019). Metrological characterization of the Intel RealSense D415 depth camera. *IEEE Transactions on Instrumentation and Measurement*, 68(11), 4386–4394. <https://doi.org/10.1109/TIM.2018.2889524>
- Daniilidis, K. (1999). Hand-eye calibration using dual quaternions. *The International Journal of Robotics Research*, 18(3), 286–298. <https://doi.org/10.1177/02783649922066213>
- Foote, E. (2013). tf: The transform library. En *IEEE International Conference on Technologies for Practical Robot Applications (TePRA)* (pp. 1–6). IEEE. <https://doi.org/10.1109/TePRA.2013.6556396>
- Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F. J., & Marín-Jiménez, M. J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6), 2280–2292. <https://doi.org/10.1016/j.patcog.2014.01.005>
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4.^a ed.). Pearson.
- Hartley, R., & Zisserman, A. (2004). *Multiple view geometry in computer vision* (2.^a ed.). Cambridge University Press.

Intel Corporation. (2023). *Intel® RealSense™ depth camera D400 series datasheet* (Revision 013). Intel Corp.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (Versión 8.0.0) [Software de computación]. Ultralytics. <https://github.com/ultralytics/ultralytics>

Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., Dollár, P., & Girshick, R. (2023). Segment anything. En *IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 4015–4026). IEEE.

Lepetit, V., Moreno-Noguer, F., & Fua, P. (2009). EPnP: An accurate $O(n)$ solution to the PnP problem. *International Journal of Computer Vision*, 81(2), 155–166. <https://doi.org/10.1007/s11263-008-0152-6>

Meta AI. (2025). *SAM 3: Promptable concept segmentation* (Technical Report). Meta AI Research.

Quigley, M., Gerkey, B., & Smart, W. D. (2015). *Programming Robots with ROS: A practical introduction to the Robot Operating System*. O'Reilly Media.

Ravi, N., Gabeur, V., Hu, Y.-T., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., Mintun, E., Pan, J., Alwala, K. V., Kirillov, A., & Feichtenhofer, C. (2024). *SAM 2: Segment Anything in images and videos* (arXiv Preprint arXiv:2408.00714). arXiv. <https://arxiv.org/abs/2408.00714>

Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2010). *Robotics: Modelling, planning and control*. Springer Science & Business Media.

Spong, M. W., Hutchinson, S., & Vidyasagar, M. (2020). *Robot modeling and control* (2.^a ed.). John Wiley & Sons.

Anexo

Anexo 1

```
#!/usr/bin/env python
import rospy
import tf
from geometry_msgs.msg import PointStamped

class DirectTransformer:
    def __init__(self):
        rospy.init_node('direct_transformer')
        self.tf_listener = tf.TransformListener()

        # Suscriptor al centroide en el sistema de coordenadas de la cámara
        self.sub_centroid = rospy.Subscriber('/object_centroid', PointStamped,
self.centroid_cb)

        # Publicador del centroide transformado al espacio del robot
```

```

        self.pub_centroid_robot = rospy.Publisher('/object_centroid_robot', PointStamped,
queue_size=1)

        self.target_frame = "m1n6s300_link_base"

def centroid_cb(self, msg):
    try:
        # Espera y consulta de la transformada desde la cámara hacia la base del robot
        self.tf_listener.waitForTransform(
            self.target_frame,
            msg.header.frame_id,
            msg.header.stamp,
            rospy.Duration(1.0)
        )

        # Aplicación de la matriz de transformación homogénea completa
        point_robot = self.tf_listener.transformPoint(self.target_frame, msg)

        # Publicación de la coordenada 3D en el marco global del robot
        self.pub_centroid_robot.publish(point_robot)
        rospy.loginfo(f"Coordenada transformada a la base: [{point_robot.point.x:.3f},
{point_robot.point.y:.3f}, {point_robot.point.z:.3f}]")

    except (tf.LookupException, tf.ConnectivityException, tf.ExtrapolationException)
as e:
        rospy.logwarn(f"Error al transformar el punto: {e}")

if __name__ == '__main__':
    transformer = DirectTransformer()
    rospy.spin()

```